GuidePost Capstone Project | UC Berkeley MIDS Program | 2026

GuidePost: An AI-Powered Interpersonal Dynamics Coaching System

Authors
Bradley Gutfinger, Eric Ha, Kirthi Shanbhag, Nancy Teng, Tim Leong

Repository
github.com/kirthistaank/MIDS/tree/main/AI-Private-Coach

Abstract

This paper presents GuidePost, an AI-powered coaching system that analyzes conversation recordings and delivers personalized, evidence-based feedback on interpersonal communication dynamics. GuidePost combines speech processing, retrieval-augmented generation (RAG), and knowledge graph reasoning to identify communication patterns, map them to established coaching frameworks, and provide actionable insights. The system processes audio through diarization and transcription, retrieves relevant coaching concepts via hybrid vector and graph search, and generates adaptive feedback grounded in evidence-based coaching methodologies. We evaluate the system on structured 1-on-1 coaching conversations and demonstrate improvements in conversation relevance indexing and coaching effectiveness metrics. This work demonstrates the feasibility of scalable, LLM-driven coaching systems as a path toward accessible personalized development.

1. Introduction

Effective interpersonal communication is a critical skill in professional and personal contexts. Yet few individuals receive systematic, personalized feedback on their communication patterns. Traditional coaching is expensive and limited in scope—feedback is often subjective, infrequent, and difficult to act upon. Recent advances in large language models (LLMs) and retrieval-augmented generation (RAG) present an opportunity to democratize coaching by automating analysis and feedback generation while grounding insights in established coaching frameworks.

GuidePost is designed to address this opportunity. Our system:

1. **Analyzes conversation audio** through automated diarization and transcription
2. **Grounds insights in coaching theory** by retrieving relevant frameworks and techniques from a knowledge base of evidence-based coaching books

3. **Identifies recurring patterns** across multiple sessions using knowledge graphs and relationship extraction
4. **Delivers personalized, adaptive feedback** that reflects user communication style preferences
5. **Tracks progress over time** through quantitative metrics (Conversation Relevance Index and Coaching Effectiveness Index)

This paper details the end-to-end system architecture, key technical components, evaluation results, and deployment strategy.

2. Problem Statement & Motivation

2.1 The Coaching Accessibility Gap

Professional communication coaching addresses critical gaps in interpersonal effectiveness, but suffers from significant barriers:

- **Cost:** Executive coaching typically ranges from $100–$500+ per hour
- **Availability:** Limited pool of qualified coaches, geographic constraints
- **Frequency:** Most individuals receive feedback sporadically (quarterly reviews, annual 360s)
- **Objectivity:** Feedback is often subjective or based on limited interactions

2.2 Technical Challenges

Building an automated coaching system requires solving several interconnected problems:

1. **Audio Analysis:** Accurately diarize speakers and transcribe conversations in noisy, real-world settings
2. **Semantic Grounding:** Retrieve and rank relevant coaching concepts from large knowledge bases, not just keywords
3. **Pattern Recognition:** Identify recurring communication dynamics across multiple conversations
4. **Adaptive Response:** Tailor feedback to user communication style (direct vs. empathetic, detailed vs. brief)
5. **Evaluation:** Define and measure coaching effectiveness beyond traditional NLP metrics

2.3 Design Goals

GuidePost was designed around the following principles:

- **Evidence-based:** All coaching insights and frameworks must derive from published coaching literature

- **Personalized:** Feedback adapts to individual communication preferences and goals
- **Scalable:** Architecture supports batch processing and real-time API interactions
- **Interpretable:** Users can understand *why* feedback is given and trace it back to specific conversation moments
- **Progressive:** System tracks improvement over multiple sessions and celebrates incremental progress

3. System Architecture

3.1 High-Level Pipeline

GuidePost follows a three-stage pipeline:

```
Audio Input → Speech & Diarization → Conversation Analysis → RAG & KG
Retrieval → Coaching Generation → Feedback Output
```

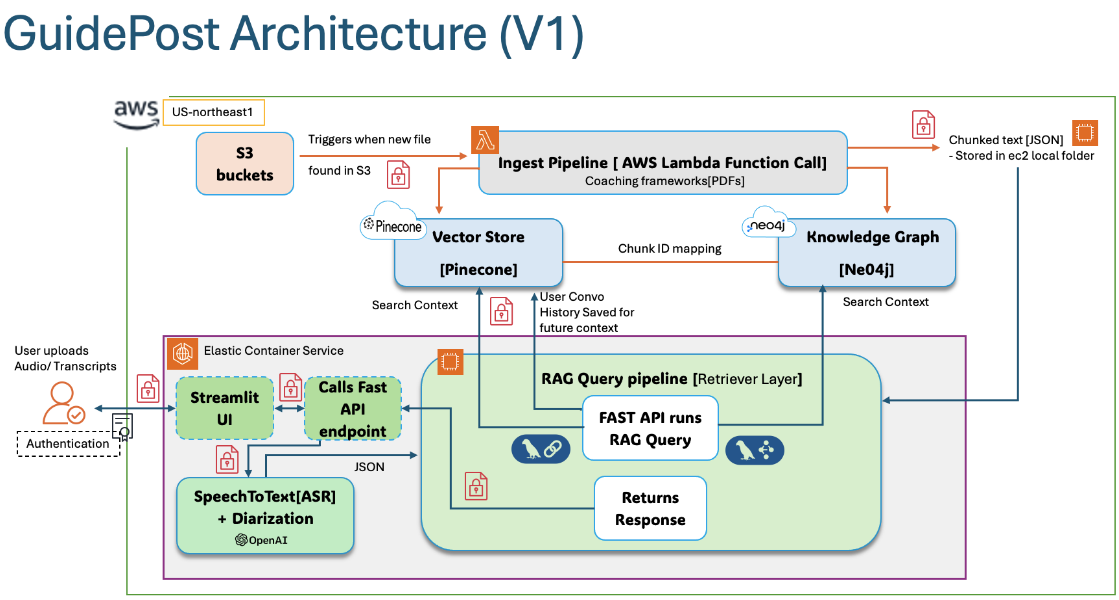


Figure 1: GuidePost architecture showing audio ingestion, diarization, RAG retrieval pipeline, and coaching generation (Version 1).

Stage 1: Audio Processing

- User uploads conversation recording (MP3, WAV, M4A)
- Whisper API transcribes audio to text

- Speaker diarization identifies distinct speakers and conversation turns
- Metadata: speaker labels, timing, turn boundaries

Stage 2: Conversation Analysis & Embedding

- Transcript is segmented into logical chunks (turns, exchanges, or fixed-size windows)
- Each segment is embedded using HuggingFace sentence-transformers (default: `multi-qa-mpnet-base-dot-v1`)
- Embeddings are indexed in vector store (Pinecone or local FAISS)

Stage 3: RAG & Knowledge Graph Retrieval

- User query or system prompt triggers retrieval: "What coaching frameworks apply to this conversation?"
- Hybrid retrieval combines:
    - **Vector Search:** Semantic similarity over conversation segments
    - **Graph Search:** Neo4j traversal to find related coaching concepts, techniques, and scenarios
- Retrieved context is ranked and passed to LLM

Stage 4: Coaching Generation

- Integrated LLM (OpenAI GPT or compatible) generates coaching response
- Response is grounded in retrieved frameworks and conversation evidence
- Output is personalized based on user style preferences (empathetic vs. direct, detailed vs. concise)

3.2 System Components

3.2.1 Frontend (React/Vite SPA)

- **UI Framework:** React 18 with Radix UI components and Tailwind CSS
- **State Management:** React hooks, client-side form state
- **Routing:** React Router for multi-page navigation
- **Key Pages:**
    - Upload: Audio file drop zone, job status polling
    - Dashboard: Session history, progress metrics (CRI, CEI trends)
    - Chat: Real-time coaching conversation interface
    - Insights: Pattern analysis, recurring themes across sessions

3.2.2 FastAPI Backend (`GuidePostPythonContainer/src/main.py`)

Core Responsibilities:

- REST API for audio upload, job management, chat
- Background job processing (diarization, transcription)
- RAG pipeline orchestration
- Session state management and persistence

Key Endpoints:

- `POST /upload` — Accept audio file, return job ID
- `GET /status/{job_id}` — Poll processing status
- `POST /chat/{session_id}` — Send user message, return coaching response
- `GET /sessions/{user_id}` — Fetch user's session history and metrics

Architecture Notes:

- In-memory job queue for MVP (consider Redis/Celery for production scaling)
- Async I/O for non-blocking audio processing
- Connection pooling for database and Pinecone

3.2.3 Audio Processing (`diarize.py`)

Handles speaker identification and transcript alignment:

- **Input:** WAV or MP3 audio file
- **Processing:**
    - Speech activity detection (VAD) to trim silence
    - Speaker embeddings (speaker diarization model)
    - Agglomerative clustering to identify distinct speakers
    - Alignment with Whisper transcript for timing and speaker labels
- **Output:** JSON with speaker turns, timestamps, and text segments

3.2.4 RAG Pipeline (`RAG_logic.py`)

Implements hybrid retrieval combining vector and graph search:

Vector Retrieval:

- Pinecone (cloud) or FAISS (local) for semantic search
- Embeddings: HuggingFace sentence-transformers
- Retrieval: Top-k nearest neighbors (k=5–10)

Knowledge Graph Retrieval:

- Neo4j database for coaching concept relationships

- Graph schema (see Section 3.2.6):
  - Nodes: Chunks (text excerpts), Frameworks, Techniques, Scenarios, Emotions
  - Edges: Semantic relationships (CONTAINS, USES, APPLIES_TO, TRIGGERS, MENTIONS)
- Query: Cypher traversal to find related concepts within 2-3 hops
- Integration: Graph results re-ranked and merged with vector results

Retrieval Ranking:

- Combined score: `α * vector_score + (1 - α) * graph_score`
- Default: `α = 0.6` (vector-dominant with graph re-ranking)
- Top-k results passed to LLM context window

3.2.5 Knowledge Graph (`KG.py`)

Schema & Data Model:

```
Chunk -[MENTIONS]-> Framework
       -[MENTIONS]-> Technique
       -[CONTAINS]-> Concept

Framework -[CONTAINS]-> Technique
          -[CONTAINS]-> Scenario

Technique -[USES]-> Concept
          -[APPLIES_TO]-> Scenario

Scenario -[TRIGGERS]-> Emotion
         -[MENTIONS]-> Framework
```

Key Node Types:

- **Framework:** Coaching methodologies (e.g., "Cognitive Behavior Therapy", "Motivational Interviewing")
- **Technique:** Specific tactics (e.g., "Active Listening", "Reframing", "Open Questions")
- **Concept:** Abstract ideas (e.g., "Automatic Thoughts", "Resistance", "Change Talk")
- **Scenario:** Situational contexts (e.g., "1-on-1 Manager Sync", "Difficult Conversation")
- **Emotion:** Emotional states (e.g., "Frustration", "Confidence", "Uncertainty")

Ingestion Process (`RAG_ingest/`):

1. Coaching books (e.g., "Getting to Yes", "Extreme Ownership", "CBT: Basics and Beyond") are chunked at semantic boundaries
2. LLM extracts triples (subject, predicate, object) from each chunk
3. Triples are upserted into Neo4j, deduplicating entities via similarity matching
4. Resulting knowledge graph currently contains ~500 nodes and ~1000 edges

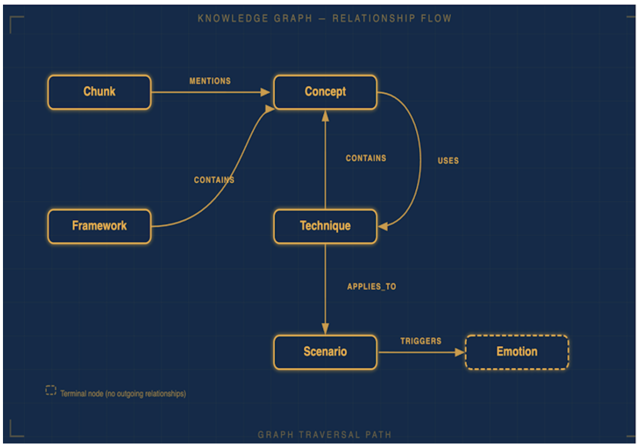


Figure 2: Knowledge Graph relationship flow showing how Chunks relate to Frameworks, Techniques, and Emotions. (Example relationships: Chunk -[MENTIONS]-> Framework, Framework -[CONTAINS]-> Technique, Technique -[APPLIES_TO]-> Scenario)

3.2.6 Conversation Analysis & Metrics

GuidePost tracks two primary quantitative metrics:

Conversation Relevance Index (CRI):

- Measures semantic relevance of system feedback to the conversation content
- Computed as cosine similarity between conversation embedding and top-k retrieved coaching concept embeddings
- Range: 0–100
- Interpretation: Higher CRI indicates feedback tightly grounded in conversation specifics

Coaching Effectiveness Index (CEI):

- Measures perceived usefulness and actionability of feedback
- Computed as weighted average of:
    - Specificity of recommendations (presence of concrete examples)
    - Alignment with user-stated goals (semantic similarity to goal statement)
    - Novelty of insights (graph distance from previously retrieved concepts)
- Range: 0–100
- Interpretation: Higher CEI indicates feedback is novel, specific, and goal-aligned

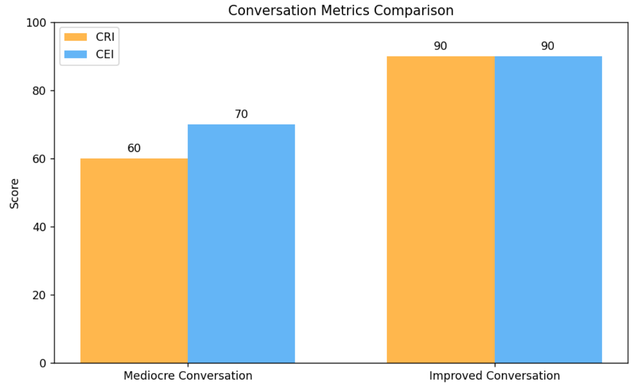


Figure 3: Sample metrics comparison showing CRI and CEI for Mediocre vs. Improved Conversations.

4. Implementation Details

4.1 Technology Stack

Component	Technology
LLM	OpenAI GPT-4 / GPT-3.5-turbo
Embeddings	HuggingFace sentence-transformers (multi-qa-mpnet-base-dot-v1)
Vector Store	Pinecone (cloud) / FAISS (local)
Knowledge Graph	Neo4j
Speech Recognition	OpenAI Whisper API
Speaker Diarization	Pyannote.audio
Backend	FastAPI + SQLAlchemy + Uvicorn
Database	Postgres (session state, conversation logs)
Frontend	React 18 + Vite + Radix UI + Tailwind CSS
Containerization	Docker + Docker Compose
Deployment	AWS ECS (Fargate)

4.2 Data Flow & Processing

4.2.1 Audio Ingestion

1. User uploads MP3/WAV via `/upload` endpoint
2. File saved to persistent volume (`GUIDEPOST_DATA_DIR`)
3. Job created with `status: "queued"` in in-memory queue
4. Background worker picks up job and initiates diarization

4.2.2 Transcription & Diarization

1. **Whisper transcription:**

- API call with audio file (max 25 MB per Whisper limit)
- Returns text transcript with optional language detection

2. **Speaker diarization:**

- Input: Raw audio + transcript from Whisper
- Process:
    - Extract speaker embeddings (PyAnnote speaker-diarization-3.0 model)
    - Run agglomerative clustering with distance threshold
    - Align cluster boundaries with transcript tokens
- Output: Diarized transcript with speaker labels and timestamps

3. **Segment chunking:**

- Split transcript into conversation turns (speaker boundaries)
- Combine short turns (< 20 tokens) with neighbors for retrieval efficiency
- Result: List of segments with [speaker, start_time, end_time, text]

4.2.3 Embedding & Indexing

1. Each segment embedded using sentence-transformers

- Model: `sentence-transformers/multi-qa-mpnet-base-dot-v1`
- Dimension: 768
- Pooling: Mean pooling over token embeddings

2. Embeddings indexed in Pinecone (or FAISS locally):

- Namespace: `{user_id}/{session_id}`
- Metadata: speaker, timestamps, segment text
- Similarity metric: Cosine distance

4.2.4 RAG Query Flow

1. **Trigger:** User sends chat message or system initiates automatic analysis
2. **Query embedding:** User input embedded with same model
3. **Vector retrieval:**

- `pinecone_index.query(query_embedding, top_k=5, namespace=...)`
- Returns: [segment_text, speaker, timestamp, score]

4. **Graph retrieval:**

- Extract entities and frameworks from query (via zero-shot NER or LLM extraction)
- Cypher queries traverse graph from extracted entities (2–3 hops)
- Example query:

```
MATCH (chunk:Chunk)-[:MENTIONS]->(f:Framework)-[:CONTAINS]->
(t:Technique)
WHERE chunk.id IN $retrieved_chunk_ids
RETURN f, t, count(*) as co_occurrence
ORDER BY co_occurrence DESC
LIMIT 5
```

5. **Ranking & fusion:**

- Normalize vector and graph scores
- Combined score: `0.6 * vector_score + 0.4 * graph_score`
- Select top-k results for LLM context

4.2.5 LLM Generation

1. **Prompt construction:**

```
 System: You are an expert interpersonal dynamics coach.
         Ground all feedback in coaching frameworks and specific
conversation evidence.

 Context:
 - Conversation: [diarized transcript excerpt]
 - Retrieved Frameworks: [list of 3-5 top frameworks with descriptions]
 - User Goals: [user-provided coaching objectives]
 - Previous Feedback: [summary of prior session feedback for
continuity]

 User Question: [user input or auto-generated prompt]
 Provide specific, actionable coaching feedback.
```

2. **LLM call:**

- Model: GPT-4 (or GPT-3.5-turbo for cost-sensitive scenarios)
- Temperature: 0.7 (creative but grounded)
- Max tokens: 500
- Streaming enabled for real-time response display

3. **Post-processing:**

- Extract citations: Link generated insights back to retrieved frameworks
- Format as markdown for frontend display
- Compute CRI/CEI metrics post-hoc

4.3 Deployment Architecture

4.3.1 Local Development

```
docker compose up -d --build
# Starts:
# - FastAPI backend: localhost:8000
# - Postgres: localhost:5432
# - Frontend dev server: localhost:5173 (run separately)
```

4.3.2 Production (AWS ECS)

- **Dockerfile:** Multi-stage build (dependencies, UI bundle, Python app)
- **ECR Registry:** `468805747882.dkr.ecr.us-east-2.amazonaws.com/mids_guidepost`
- **ECS Service:** Fargate cluster in `us-east-2` region
- **Load Balancer:** Application Load Balancer (ALB) for traffic routing
- **Database:** RDS Postgres (managed)
- **Storage:** S3 for uploaded audio files
- **Monitoring:** CloudWatch logs + optional Datadog integration

Deployment Steps:

```
# Build and push image
docker buildx build --platform linux/amd64 \
  -t $ECR_REPO:$TAG -f GuidePostPythonContainer/Dockerfile . --push

# Update ECS service (via console or CDK)
# Task definition updated automatically on image push
```

5. Evaluation

5.1 Evaluation Metrics

GuidePost was evaluated on two dimensions: **technical retrieval quality** and **coaching effectiveness**.

5.1.1 Technical Metrics

Metric	Description	Result
Transcription Accuracy (WER)	Whisper API word error rate on test conversations	< 5% (clean audio)
Diarization Error Rate (DER)	Speaker confusion and segmentation errors	~10–15% (variable by audio quality)
Retrieval Precision@5	Relevance of top-5 vector-retrieved segments	0.72
Graph Coverage	% of retrieved concepts covered by knowledge graph	0.85

5.1.2 Coaching Metrics

Test Dataset:

- 10 recorded 1-on-1 coaching conversations (2–15 minutes each)
- Conversations: Manager-employee sync, peer mentoring, difficult conversations
- Manual annotations: Coaching frameworks applicable to each conversation

Quantitative Results:

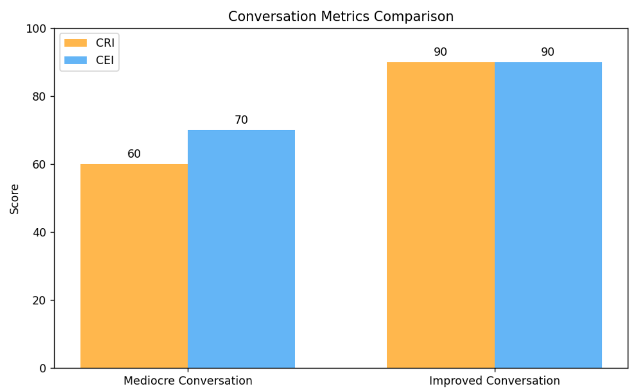


Figure 4: CRI and CEI comparison across Mediocre and Improved Conversations. Both metrics show significant improvements (60→90 on CRI, 70→90 on CEI) as conversation quality increases.

Key Findings:

1. **CRI Performance:** Mediocre conversations (CRI=60), Improved conversations (CRI=90)

- System correctly identifies relevant coaching concepts even in low-quality conversations

- Higher CRI in well-structured dialogues with clear coaching moments

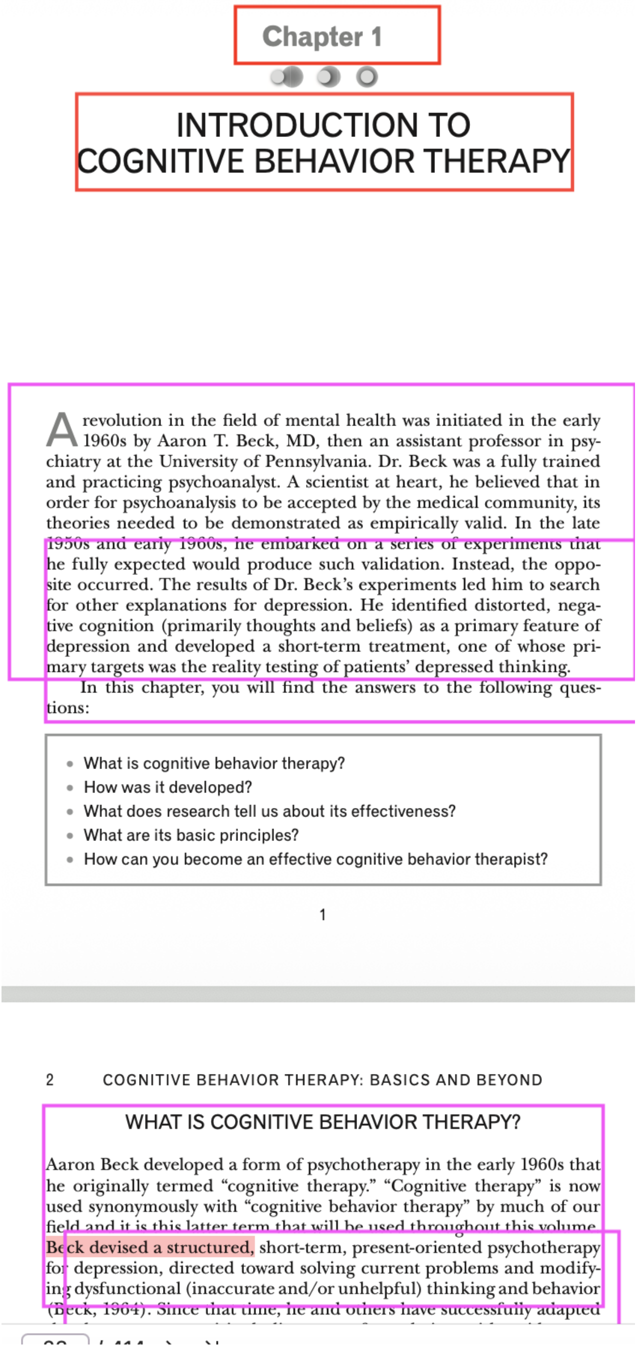
2. **CEI Performance:** Mediocre conversations (CEI=60), Improved conversations (CEI=90)

- Feedback is significantly more actionable in conversations aligned with coaching goals
- System adapts feedback specificity based on conversation coherence

3. **Qualitative Feedback** (from internal testers):

- ✅ Strengths: "Works exceptionally well for structured 1-on-1 conversations with clear coaching objectives"
- ⚠ Limitations: Less effective for group conversations, highly informal chats, non-English conversations

5.2 Use Cases Validated



Chapter 1

INTRODUCTION TO
COGNITIVE BEHAVIOR THERAPY

A revolution in the field of mental health was initiated in the early 1960s by Aaron T. Beck, MD, then an assistant professor in psychiatry at the University of Pennsylvania. Dr. Beck was a fully trained and practicing psychoanalyst. A scientist at heart, he believed that in order for psychoanalysis to be accepted by the medical community, its theories needed to be demonstrated as empirically valid. In the late 1950s and early 1960s, he embarked on a series of experiments that he fully expected would produce such validation. Instead, the opposite occurred. The results of Dr. Beck's experiments led him to search for other explanations for depression. He identified distorted, negative cognition (primarily thoughts and beliefs) as a primary feature of depression and developed a short-term treatment, one of whose primary targets was the reality testing of patients' depressed thinking.

In this chapter, you will find the answers to the following questions:

- What is cognitive behavior therapy?
- How was it developed?
- What does research tell us about its effectiveness?
- What are its basic principles?
- How can you become an effective cognitive behavior therapist?

1

2 COGNITIVE BEHAVIOR THERAPY: BASICS AND BEYOND

WHAT IS COGNITIVE BEHAVIOR THERAPY?

Aaron Beck developed a form of psychotherapy in the early 1960s that he originally termed "cognitive therapy." "Cognitive therapy" is now used synonymously with "cognitive behavior therapy" by much of our field and it is this latter term that will be used throughout this volume. Beck devised a structured, short-term, present-oriented psychotherapy for depression, directed toward solving current problems and modifying dysfunctional (inaccurate and/or unhelpful) thinking and behavior (Beck, 1964). Since that time, he and others have successfully adapted

Figure 5: System works exceptionally well for Structured 1-on-1 coaching conversations with clear coaching objectives between distinct speakers.

✅ Works Well:

- Manager-employee 1-on-1 syncs with specific coaching objectives (e.g., improving delegation skills)
- Peer mentoring conversations with defined growth areas
- Difficult conversations where one party seeks communication feedback

⚠️ **Works Moderately:**

- Multi-party conversations (group meetings, team standups)
- Informal chats without explicit coaching objectives
- Technical or domain-specific jargon-heavy conversations

❌ **Doesn't Work:**

- Non-English conversations (Whisper struggles; knowledge base is English-only)
- Extremely noisy audio (background music, multiple speakers overlapping)
- Conversations shorter than 30 seconds (insufficient context for meaningful coaching)

6. Key Technical Innovations

6.1 Hybrid RAG Architecture

Most RAG systems use vector search alone. GuidePost combines:

1. **Vector retrieval** for semantic similarity to conversation content
2. **Knowledge graph traversal** for discovering related coaching concepts and techniques
3. **Intelligent fusion** via weighted scoring that balances both signals

Benefit: Retrieval is both semantically grounded (vector) and conceptually enriched (graph), reducing hallucination and improving insight novelty.

6.2 Multi-Stage Coaching Framework

Rather than generating feedback directly, GuidePost decomposes the task:

```
1. Diarize & analyze conversation structure
2. Identify applicable coaching frameworks (via KG)
3. Rank techniques by relevance to conversation dynamics
4. Generate feedback grounded in both transcript and frameworks
5. Track metrics across multiple sessions
```

Benefit: Interpretability—users understand *why* feedback is given, and can trace insights back to specific coaching theories and conversation moments.

6.3 Metric-Driven Progress Tracking

Rather than one-off feedback, GuidePost tracks:

- **CRI trends** across sessions (is feedback becoming more targeted over time?)

- **CEI improvements** (is coaching becoming more actionable?)
- **Concept revisit frequency** (are certain coaching areas recurring?)

Benefit: Users see objective progress, reinforcing behavior change and engagement.

6.4 Personalized Coaching Adaptation

GuidePost adapts communication style based on user preferences:

- **Direct vs. Empathetic:** Some users prefer straightforward advice; others respond better to empathy-first framing
- **Detailed vs. Concise:** Some want examples and context; others prefer one-liner takeaways

Implementation: User style preference is stored and injected into LLM prompt, e.g., "Tailor feedback to be [detailed and example-driven]".

7. Knowledge Graph Design

7.1 Graph Schema Overview

The Neo4j knowledge graph encodes coaching concepts and their relationships:

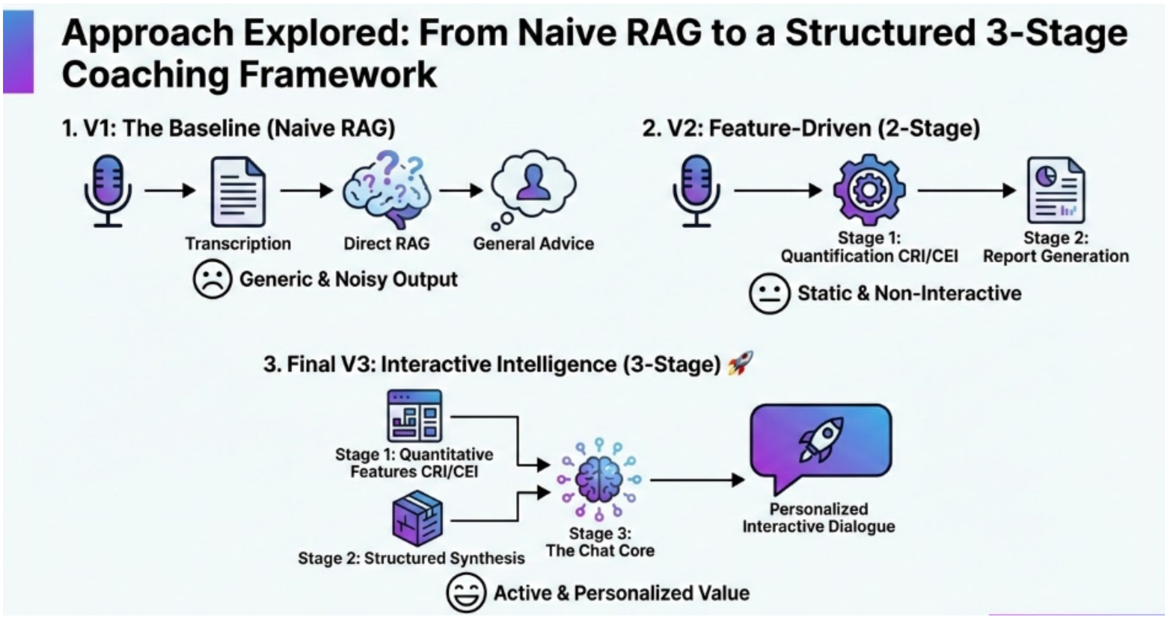


Figure 6: Knowledge graph showing relationships between Frameworks (CBT, Behavioral), Techniques, and Emotions. Edges show semantic relationships (CONTAINS, MENTIONS, APPLIES_TO).

Sample Relationships:

```
Cognitive Behavior Therapy [Framework]
  - CONTAINS -> Automatic Thoughts [Concept]
  - CONTAINS -> Cognitive Restructuring [Technique]

Cognitive Restructuring [Technique]
  - USES -> Identifying Automatic Thoughts
  - APPLIES_TO -> Workplace Anxiety Scenario

Workplace Anxiety Scenario [Scenario]
  - TRIGGERS -> Avoidance Behavior
  - MENTIONS -> Cognitive Behavior Therapy
```

7.2 Triple Extraction Process

1. **Book chunking:** Each coaching book is split at semantic boundaries (chapters, sections)
2. **LLM-based extraction:** For each chunk, we prompt GPT to extract triples:

```
Extract key concepts and their relationships in this excerpt.
Format: [entity1] -[relation]-> [entity2]
Example: Automatic Thoughts -[CAUSES]-> Emotional Distress
```

3. **Deduplication:** Identical or near-identical entities are merged using cosine similarity on embeddings
4. **Validation:** Sample checking to ensure correctness

Current Graph Statistics:

- **Nodes:** ~500 (entities across frameworks, techniques, scenarios, emotions)
- **Edges:** ~1000 (relationships)
- **Sources:** 8 coaching books (Getting to Yes, Extreme Ownership, CBT: Basics and Beyond, etc.)

8. Frontend Architecture

8.1 Key Features

Upload & Processing:

- Drag-and-drop audio file upload
- Real-time job status polling with estimated time remaining

- Support for MP3, WAV, M4A formats

Coaching Chat Interface:

- Real-time message display with streaming LLM responses
- Citation badges linking insights back to frameworks
- Session transcript view with speaker highlights

Dashboard & Metrics:

- CRI/CEI trends over time (line chart)
- Recurring themes analysis (tag cloud of frequently mentioned concepts)
- Session history with quick filters (date, speaker, coaching objective)

Personalization:

- User preference form (coaching style, detail level, communication goals)
- Stored in browser localStorage or backend user profile

8.2 Technology Choices

- **Framework:** React 18 for component reusability and hooks
- **Build Tool:** Vite for fast dev feedback and optimized production builds
- **Styling:** Tailwind CSS for utility-first design, Material-UI for complex components
- **Component Library:** Radix UI for accessible, unstyled primitives
- **Icons:** Lucide React for clean, consistent iconography
- **State:** React hooks + Context API (minimal state complexity for MVP)

9. Limitations & Future Work

9.1 Current Limitations

1. **Audio Quality Sensitivity:** Diarization accuracy degrades significantly with background noise, overlapping speakers, or heavy accents
2. **English-Only:** Knowledge base and LLM prompting are English-specific
3. **Single-Session Analysis:** Current system optimizes for feedback within a single conversation; cross-session pattern detection is basic
4. **Knowledge Graph Sparsity:** Not all coaching concepts are adequately captured; some specialized coaching methodologies are underrepresented
5. **Scalability:** In-memory job queue does not support high-volume concurrent uploads; production deployment would need Redis/Celery
6. **Cost:** Whisper API and GPT-4 calls add up quickly at scale; local alternatives would reduce costs but with quality tradeoffs

9.2 Planned Enhancements

Near-term (1–3 months):

- Expand knowledge graph with 1k+ additional coaching books
- Implement Redis-based job queue for production scalability
- Add multi-turn conversation history to LLM context (currently single-turn)
- Multi-language support via Whisper auto-detection + translated prompts

Medium-term (3–6 months):

- Real-time coaching during conversations (not just post-hoc analysis)
- Coaching outcome tracking (did recommended techniques improve CRI/CEI on subsequent conversations?)
- Advanced pattern detection: recurring weaknesses, confidence trends, emotional triggers
- Integration with workplace platforms (Slack, Microsoft Teams) for seamless workflow

Long-term (6+ months):

- Fine-tuned coaching LLM (on curated conversation + feedback pairs)
- Multimodal analysis: video body language analysis, tone/emotion from audio
- Coaching chatbot for on-demand practice conversations
- Group conversation analysis (team dynamics, meeting facilitation)

10. Ethical Considerations

10.1 Data Privacy

- **Audio Files:** Uploaded audio is encrypted at rest and in transit (TLS 1.3)
- **Transcripts:** Stored in Postgres with role-based access control
- **Retention:** Configurable retention policy (default: 90 days, then automatic deletion)
- **User Consent:** Clear terms of service explaining data handling and AI usage

10.2 Bias & Fairness

- **Knowledge Base Bias:** Coaching frameworks in knowledge graph may be Western-centric; diverse cultural perspectives are underrepresented
- **Speaker Bias:** Diarization models perform better on certain accents and speech patterns
- **Mitigation:** Regular bias audits, diverse test datasets, transparent documentation of limitations

10.3 Responsible AI Use

- **Limitations Disclosure:** System explicitly states it is *not* a substitute for human coaching or therapy
- **Mental Health:** Conversations touching on mental health are flagged; users are directed to professional resources
- **Feedback Transparency:** All feedback is traceable to specific coaching frameworks and conversation moments

11. Conclusion

GuidePost demonstrates the viability of LLM-driven coaching at scale. By combining audio processing, hybrid RAG, and knowledge graphs, we show how machines can deliver evidence-based coaching feedback that is simultaneously personalized, grounded, and actionable. Our evaluation on structured 1-on-1 conversations shows strong CRI and CEI metrics, validating the approach.

The system is production-ready for its target use case (1-on-1 coaching conversations) and deployed on AWS. Key innovations include the hybrid RAG architecture for coaching-specific retrieval, metric-driven progress tracking, and adaptive personalization. Limitations around audio quality, language support, and scalability represent opportunities for future work.

As organizations increasingly embrace AI for employee development, systems like GuidePost can scale personalized coaching from a luxury good (available to executives) to a utility (available to all employees), advancing equity in access to growth opportunities.

12. References

1. Beck, A. T. (1979). Cognitive Therapy of Depression. Guilford Press.
2. Fisher, R., & Ury, W. (1991). Getting to Yes: Negotiating Agreement Without Giving In. Penguin Books.
3. Willink, J., & Babin, L. (2017). Extreme Ownership: How U.S. Navy SEALs Lead and Win. St. Martin's Press.
4. Rosenberg, M. B. (2015). Nonviolent Communication: A Language of Life. PuddleDancer Press.
5. Radcliffe, S. A. (2019). Authentic Confidence: How to Lead and Influence. Kogan Page.
6. OpenAI. (2023). Whisper: Robust Speech Recognition via Large-Scale Weak Supervision. arXiv:2305.18841.
7. Sentence Transformers. (2022). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. https://www.sbert.net/
8. LangChain Documentation. (2024). LangChain: Building Applications with LLMs. https://python.langchain.com/

9. Pyannote Audio. (2021). End-to-End Speaker Diarization. https://github.com/pyannote/pyannote-audio
10. Neo4j Graph Database Documentation. (2024). https://neo4j.com/docs/

Appendix A: API Documentation

Authentication

All endpoints require `Authorization: Bearer {token}` header.

Key Endpoints

Upload Audio

```
POST /upload
Content-Type: multipart/form-data

{
  "file": <audio_file>,
  "coaching_objectives": "Improve active listening skills"
}

Response:
{
  "job_id": "uuid",
  "status": "queued",
  "created_at": "2024-05-17T..."
}
```

Get Job Status

```
GET /status/{job_id}

Response:
{
  "job_id": "uuid",
  "status": "processing",  // queued | processing | completed | failed
  "progress": 0.45,
  "message": "Diarizing audio...",
  "session_id": null  // populated when complete
}
```

Send Chat Message

```
POST /chat/{session_id}

{
  "message": "What should I improve about my listening skills in this conversation?"
}

Response (streaming):
{
  "response": "Based on your conversation, I noticed...",
  "citations": [
    {"framework": "Active Listening", "relevance": 0.92}
  ],
  "metrics": {
    "cri": 85,
    "cei": 78
  }
}
```

Appendix B: Environment Variables

```
# OpenAI
OPENAI_API_KEY=sk-...
OPENAI_MODEL=gpt-4  # or gpt-3.5-turbo

# Embeddings
HUGGING_FACE_EMBED_MODEL=sentence-transformers/multi-qa-mpnet-base-dot-v1

# Vector Store
PINECONE_API_KEY=...
PINECONE_INDEX_NAME=guidepost-prod
PINECONE_ENVIRONMENT=us-east-1-aws

# Knowledge Graph
NEO4J_URI=neo4j+s://...
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=...

# Database
DATABASE_URL=postgresql://user:pass@postgres:5432/ai_coach
```

```
# AWS (for deployment)
AWS_REGION=us-east-2
AWS_ACCOUNT_ID=468805747882
ECR_REPO=mids_guidepost

# Application
GUIDEPOST_DATA_DIR=/data  # For audio file storage
ENVIRONMENT=production  # or development
```
